Studio Nx

Backend(FastAPI/Python) 15 — hash_password()

2026-10-12

Backend(FastAPI/Python) 15 — `hash_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입: 비밀번호를 해시로 저장 (PRD 01 FR-102)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	14 `check_password_rules()` · 16 `verify_password()`

1. 하는 일

비밀번호를 **되돌릴 수 없는 문자열(해시)**로 바꿉니다. DB에는 비밀번호 대신 이 해시만 저장합니다.

`hash_password("Abcdef1!")` → `"$2b$12$Q9xK…(60자)"`

해시의 부분	예	뜻
`$2b$`		bcrypt 방식이라는 표시입니다
`12$`		비용(cost) 12입니다. 계산을 2^{12}번 반복하며, 클수록 느립니다
다음 22자	`Q9xKc3…`	소금(salt)입니다. 해시마다 새로 만든 무작위 값입니다
나머지 31자		비밀번호와 소금으로 계산한 결과입니다

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (21)	규칙 검사(14)를 통과한 비밀번호를 해시해서 `users.password_hash`에 저장합니다
P1-09 `change_password()`	새 비밀번호를 해시해서 바꿔 저장합니다

로그인(P1-05)할 때는 이 해시를 되돌리지 않습니다. 입력한 비밀번호를 같은 소금으로 다시 계산해서 결과가 같은지 비교합니다 (다음 함수 16 `verify_password()`).

비밀번호를 그대로 저장하지 않는 이유

저장 방식	DB가 유출되면
비밀번호 그대로	모든 회원의 비밀번호가 바로 드러납니다. 같은 비밀번호를 쓰는 다른 사이트까지 위험해집니다
단순 해시 (SHA-256 등)	계산이 빨라서, 흔한 비밀번호를 초당 수십억 개씩 대입해 찾아낼 수 있습니다
bcrypt (소금 + 느린 계산)	한 번 계산하는 데 약 0.2초가 걸립니다. 또 회원마다 소금이 달라서 미리 만들어 둔 표로 찾을 수 없습니다

3. 코드

import (파일 맨 위)

```
import re
import string

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다. **`import bcrypt`를 이번에 추가했고**, 이 함수는 그것만 사용합니다. `bcrypt` 패키지는 `requirements.txt`에 이미 있습니다 (설치된 버전 5.0.0).

함수

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("ascii")
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름`으로 씁니다 (예: `bcrypt.hashpw`)

한 줄짜리 코드를 안쪽부터 풀면 다음과 같습니다.

순서	식	값
①	`plain.encode("utf-8")`	`"Abcdef1!"`를 `b"Abcdef1!"`로 바꿉니다 (문자열 → 바이트. bcrypt는 바이트만 받습니다)
②	`bcrypt.gensalt()`	새 소금 `b"$2b$12$Q9xKc3…"`을 만듭니다 (기본 비용 12)
③	`bcrypt.hashpw(①, ②)`	해시 `b"$2b$12$Q9xKc3…(60바이트)"`를 계산합니다 (바이트)
④	`.decode("ascii")`	바이트를 문자열로 바꿉니다. DB의 `VARCHAR` 칸에 넣기 위해서입니다

식	뜻
`b"…"`	바이트 값입니다. 문자열(`str`)과 다른 타입입니다
`"ascii"`로 되돌림	bcrypt 해시는 영문, 숫자, `$./`만 쓰므로 ASCII로 충분합니다

4. 흐름

1. 비밀번호를 바이트로 바꿉니다.
2. 새 소금을 만듭니다.
3. 비밀번호와 소금으로 해시를 계산합니다 (약 0.2초).
4. 문자열로 바꿔서 돌려줍니다.

5. 왜 이렇게

선택	이유
bcrypt 사용	PRD FR-102가 정한 방식입니다. 오랫동안 검증된 비밀번호 전용 해시입니다
소금을 매번 새로 만듦	같은 비밀번호라도 회원마다 해시가 달라집니다. 한 사람의 비밀번호를 알아내도, 같은 비밀번호를 쓰는 다른 회원은 드러나지 않습니다

선택	이유
소금을 따로 저장하지 않음	해시 안(`$2b$12$` 뒤 22자)에 들어 있으므로, 칸 하나(`password_hash`)로 충분합니다
비용 12 (기본값)	이 컴퓨터에서 한 번에 약 0.19초가 걸립니다. 가입이나 로그인 한 번에는 느끼지 못하지만, 대입 공격에는 매우 느립니다
72바이트 제한을 따로 처리하지 않음	bcrypt 5는 72바이트가 넘으면 오류를 냅니다. 비밀번호는 14번에서 최대 20자이고 영문, 숫자, 기호(1글자 = 1바이트)만 쓰도록 검사한 뒤 들어오므로, 72바이트를 넘을 수 없습니다

6. 시험 (`tests/test_password.py`에 5개를 추가해서 모두 21개)

시험	확인한 것
모양	문자열이고, `$2b$12$`로 시작하며, 60자입니다 (`VARCHAR(255)`에 충분합니다)
매번 다름	같은 비밀번호를 두 번 해시하면 결과가 다릅니다 (소금 때문입니다)
원래 비밀번호만 맞음	`bcrypt.checkpw`로 비교하면 `"Abcdef1!"`은 맞고 `"Abcdef1?"`는 틀립니다
비밀번호가 보이지 않음	해시 안에 원래 글자가 들어 있지 않습니다
가장 긴 비밀번호 (20자)	오류 없이 해시되고, 비교도 맞습니다

비교 시험은 다음 함수(16 `verify_password`) 대신 `bcrypt.checkpw`를 직접 썼습니다.

전체 시험 97개가 통과했고, `ruff` 검사도 통과했습니다. 해시 한 번에 약 190ms가 걸립니다.